

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

ассистент

должность, уч. степень, звание

подпись, дата

Д.О. Шевяков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №3

Архитектура классов

по курсу: Основы программирования

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

СОДЕРЖАНИЕ

1 ЦЕЛЬ РАБОТЫ.....	3
2 ЗАДАНИЕ	4
3 ОПИСАНИЕ РАБОТЫ ПРОГРАММЫ	5
4 ИСХОДНЫЙ КОД ПРОГРАММЫ	8
5 РЕЗУЛЬТАТ РАБОТЫ ПРОГРАММЫ	14
6 ВЫВОД.....	15

1 Цель работы

Изучить свойства классов get и set, реализовать их в приложении Windows Forms.

2 Задание

Разработать для одного из символьных свойств механизм проверки: является ли символ введённого значения прописным или строчным, если строчным, то его следует заменить прописным.

Разработать свойства для полей с различными типами данных: числовых, дата-время, строковых.

3 Описание работы программы

В данной реализации класса Car предусмотрено несколько усовершенствований для обработки различных типов данных и обеспечения корректности работы с ними. Свойство Symbol выполняет проверку регистра символа: если введенный символ является строчной буквой, он автоматически преобразуется в прописную. Это обеспечивает унифицированный подход к обработке данных, независимо от их исходного формата.

```
private char _symbol;
Ссылка: 0
public char Symbol
{
    get => _symbol;
    set => _symbol = char.IsLower(value) ? char.ToUpper(value) : value;
}

// Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность
private DateTime _lastServiceDate;
Ссылка: 0
public DateTime LastServiceDate
{
    get => _lastServiceDate;
    set
    {
        if (value < DateTime.Now)
            throw new ArgumentException("Дата не может быть в прошлом.");
        _lastServiceDate = value;
    }
}
```

```
Ссылка: 9
public string Condition { get; set; } = "Новое"; // Состояние колеса
Ссылка: 2
public int BrokenCount { get; set; } = 0; // Счетчик поломок колеса
```

```
Ссылка: 3
public bool IsRunning { get; private set; }

// Метод Start запускает двигатель, если его состояние не "Сломано"
// Если двигатель сломан, выбрасывается исключение
Ссылка: 1
public void Start()
{
    if (Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, его нельзя запустить.");
    IsRunning = true;
}
```

Рисунок 1, 2, 3 – Проверка класса и Времени формы и пример модификатора метода get set

Для свойства Fuel добавлена проверка, предотвращающая присвоение отрицательных значений, что исключает логические ошибки и делает использование класса более надёжным. Также реализовано свойство Name, которое автоматически удаляет лишние пробелы и выбрасывает исключение в случае пустого значения, гарантируя корректность ввод.

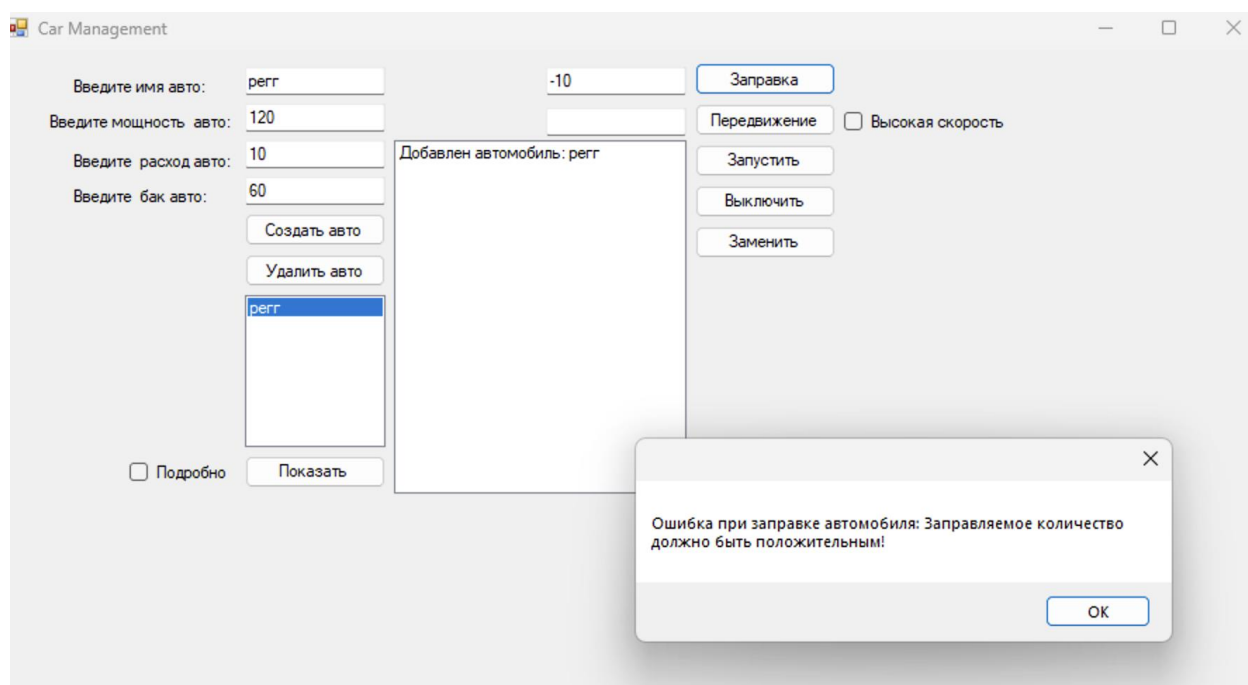


Рисунок 4 – Проверка объема бензина

В дополнение к этому, в классе реализовано свойство LastServiceDate, которое отвечает за хранение даты последнего обслуживания автомобиля. Проверка на корректность данных предотвращает ввод дат, которые находятся в прошлом, что важно для управления техническим состоянием автомобиля. Метод GetCarInfo был доработан, чтобы выводить информацию о текущем состоянии автомобиля, включая данные о двигателе, пробеге, уровне топлива и убрана лишняя проверка данных для вывода, что упростила код.

```

// Метод для получения информации о колесе
Ссылка: 0
public string GetInfo(bool showDetailed)
{
    // Базовая информация о состоянии колеса
    string info = $"Колесо состояние: {Condition}";

    if (showDetailed)
    {
        // Если запрашиваются подробности, добавляем информацию о количестве поломок
        info += $"{Environment.NewLine}Количество поломок: {BrokenCount}";
    }

    return info; // Возвращаем итоговую информацию
}

```

Рисунок 5 – Облегчен метод GetInfo

Такая реализация делает класс универсальным и удобным для использования в различных сценариях, связанных с управлением автомобилями.

4 Исходный код программы

Ниже представлен исходный код программы.

Файл Car.cs:

```
using System;
using System.Linq;

namespace CarManagementApp
{
    // Класс Car представляет автомобиль с его компонентами (двигатель, колеса) и управлением ими
    public class Car
    {
        // Свойство Name хранит название автомобиля с обработкой лишних пробелов
        private string _name;
        public string Name
        {
            get => _name;
            set => _name = value?.Trim() ?? throw new ArgumentException("Название не может быть пустым.");
        }

        // Свойство Engine хранит объект двигателя автомобиля
        public Engine Engine { get; set; }

        // Свойство Wheels хранит массив колес автомобиля (массив из 4 колес)
        public Wheel[] Wheels { get; set; }

        // Свойство Fuel хранит текущее количество топлива в автомобиле с проверкой на отрицательные значения
        private int _fuel;
        public int Fuel
        {
            get => _fuel;
            set
            {
                if (value < 0)
                    throw new ArgumentException("Количество топлива не может быть отрицательным.");
                _fuel = value;
            }
        }

        // Свойство FuelTankCapacity хранит максимальный объем топливного бака
        public int FuelTankCapacity { get; private set; }

        // Свойство Mileage хранит пробег автомобиля в километрах
        public int Mileage { get; private set; } = 0;

        // Свойство FuelConsumption хранит расход топлива на 100 км
        public int FuelConsumption { get; private set; }

        // Статическое свойство для общего количества колес
        public static int WheelCount { get; } = 4;

        // Свойство Symbol проверяет регистр символа и преобразует строчные буквы в прописные
        private char _symbol;
        public char Symbol
        {
            get => _symbol;
            set => _symbol = char.IsLower(value) ? char.ToUpper(value) : value;
        }
    }
}
```



```

// Свойство LastServiceDate хранит дату последнего обслуживания с проверкой на корректность
private DateTime _lastServiceDate;
public DateTime LastServiceDate
{
    get => _lastServiceDate;
    set
    {
        if (value < DateTime.Now)
            throw new ArgumentException("Дата не может быть в прошлом.");
        _lastServiceDate = value;
    }
}

// Конструктор класса Car
// Инициализирует автомобиль с заданными параметрами: имя, мощность двигателя, расход топлива и
объем бака
public Car(string name, int horsepower, int fuelConsumption, int fuelTankCapacity)
{
    Name = name;

    Engine = new Engine { HorsePower = horsepower };
    Wheels = new Wheel[4]; // Инициализация массива колес

    // Инициализация колес и подписка на событие поломки каждого колеса
    for (int i = 0; i < WheelCount; i++)
    {
        Wheels[i] = new Wheel();
        Wheels[i].ComponentBroken += OnComponentBroken; // Подписка на событие поломки колеса
    }

    Engine.ComponentBroken += OnComponentBroken; // Подписка на событие поломки двигателя

    FuelConsumption = fuelConsumption;
    FuelTankCapacity = fuelTankCapacity;
    Fuel = FuelTankCapacity; // Заполнение бака до максимума
}

// Свойство IsEngineOn проверяет, работает ли двигатель
// Возвращает true, если двигатель включен, иначе false
public bool IsEngineOn => Engine.IsRunning;

// Метод StartEngine запускает двигатель
public void StartEngine() => Engine.Start();

// Метод StopEngine выключает двигатель
public void StopEngine() => Engine.Stop();

// Обработчик события поломки компонента (колеса или двигателя)
// При поломке компонента выводится сообщение о поломке в консоль
private void OnComponentBroken(object sender, EventArgs e)
{
    if (sender is Engine)
    {
        // Сообщение о поломке двигателя
        Console.WriteLine($"{Name}: Двигатель сломан!");
    }
    else if (sender is Wheel wheel)
    {
        // Сообщение о поломке колеса
    }
}

```

```

        Console.WriteLine($"{Name}: Колесо {Array.IndexOf(Wheels, wheel) + 1} сломано!");
    }
}

// Перегрузка метода Move: стандартное движение (обычная скорость)
// Этот метод вызывает второй метод Move с параметром isFast = false (стандартная скорость)
public void Move(int distance)
{
    Move(distance, false); // Перемещение с обычной скоростью
}

// Перегрузка метода Move: движение с дополнительными параметрами (например, высокая скорость)
// Если параметр isFast равен true, расход топлива снижается на 20% (ускоренный режим)
public void Move(int distance, bool isFast = false)
{
    // Проверка, включен ли двигатель, если выключен - нельзя двигаться
    if (!IsEngineOn)
    {
        throw new InvalidOperationException("Двигатель выключен! Невозможно передвигаться.");
    }

    // Проверка, не сломан ли двигатель
    if (Engine.Condition == "Сломано")
        throw new InvalidOperationException("Двигатель сломан, машина не может двигаться.");

    // Проверка, достаточно ли топлива для движения
    if (Fuel < (distance * FuelConsumption) / 100)
        throw new InvalidOperationException("Недостаточно топлива!");

    // Проверка, не сломано ли одно из колес
    if (Wheels.Any(w => w.Condition == "Сломано"))
        throw new InvalidOperationException("Одно из колес сломано!");

    // Увеличиваем пробег
    Mileage += distance;

    // Расчет расхода топлива в зависимости от скорости
    if (isFast)
    {
        // Ускоренная скорость: расход топлива снижается на 20%
        Fuel -= (int)((distance * FuelConsumption) / 100 * 0.8); // Приведение типа к int
    }
    else
    {
        // Обычная скорость: расход топлива без изменений
        Fuel -= (distance * FuelConsumption) / 100;
    }

    // Поломка колеса каждые 200 км
    if (Mileage % 200 == 0)
    {
        Random random = new Random();
        int brokenWheelIndex = random.Next(Wheels.Length);
        Wheels[brokenWheelIndex].ChangeCondition("Сломано"); // Случайная поломка одного из колес
    }

    // Поломка двигателя каждые 1000 км
    if (Mileage % 1000 == 0)
    {
        Engine.ChangeCondition("Сломано"); // Поломка двигателя после 1000 км пробега
    }
}

```

```

// Метод Refuel заправляет автомобиль на указанное количество литров
// Проверяет, не превышает ли заправка емкость бака
public void Refuel(int amount)
{
    if (amount <= 0)
        throw new ArgumentException("Заправляемое количество должно быть положительным!");

    Fuel += amount;
    if (Fuel > FuelTankCapacity)
        Fuel = FuelTankCapacity; // Если заправлено больше, чем вмещает бак, устанавливаем максимальную
емкость
}

// Метод ReplaceBrokenWheel заменяет сломанное колесо на новое
public void ReplaceBrokenWheel()
{
    for (int i = 0; i < Wheels.Length; i++)
    {
        if (Wheels[i].Condition == "Сломано")
        {
            Wheels[i].ChangeCondition("Новое"); // Заменить сломанное колесо
            return;
        }
    }
    throw new InvalidOperationException("Нет сломанных колес для замены."); // Если нет сломанных колес
}

// Метод для получения информации о машине
// Если параметр showDetailed = true, выводится подробная информация
public virtual string GetCarInfo(bool showDetailed = false)
{
    // Основная информация о машине
    string info = $"Название: {Name}\n" +
        $"Мощность двигателя: {Engine.HorsePower} л.с.\n" +
        $"Пробег: {Mileage} км\n" +
        $"Топливо: {Fuel}/{FuelTankCapacity} л\n" +
        $"Расход топлива: {FuelConsumption} л/100 км\n" +
        $"Двигатель: {(IsEngineOn ? "Включен" : "Выключен")} ({Engine.Condition})\n" +
        $"Колеса: {string.Join(", ", Wheels.Select((w, i) => $"Колесо {i + 1}: {w.Condition}"))}";

    // Если showDetailed == true, добавляется подробная информация
    if (showDetailed)
    {
        info += $"Оставшийся пробег до поломки двигателя: {1000 - Mileage % 1000} км";
        for (int i = 0; i < Wheels.Length; i++)
        {
            info += $"Колесо {i + 1}: осталось до поломки {200 - Mileage % 200} км";
        }
    }

    return info;
}
}

```

Файл Form1.cs:

```

using System;
using System.Windows.Forms;
using myliba;

namespace op3

```

```

{
    public partial class Form1 : Form
    {
        public Form1()
        {
            InitializeComponent();

            DayOfWeekToRussian day = new DayOfWeekToRussian(); // день недели на русском
            day.Dw = DateTime.Now.DayOfWeek.ToString();

            monthNameToRussian month = new monthNameToRussian(); // месяц на русском
            month.M = DateTime.Now.Month.ToString();

            string dayofmonth = DateTime.Now.Day.ToString(); // сегодняшний день месяца

            string year = DateTime.Now.Year.ToString(); // текущий год

            dtLabel.Text = string.Format("{0}, {1} {2} {3}", // формат: среда, 1 января 1970
                day.Dw, dayofmonth, month.M, year);

            operationName.SelectedItem = "Закупка";
            itemName.SelectedItem = "куртка";
        }

        private void writeButton_Click(object sender, EventArgs e)
        {
            string addinfo = operationName.Text == "Закупка" ? AddInfo.BuyText : AddInfo.SellText;

            Supply supply;
            if (brandName.Text == "")
            {
                supply = new Supply(operationName.Text, itemName.Text, (int)count.Value);
            }
            else
            {
                supply = new Supply(operationName.Text, itemName.Text, brandName.Text, (int)count.Value);
            }

            writeResultField.Text += string.Format("{0}: {1} бренда {2} в кол-ве {3} шт. {4}\n",
                supply.Operation, supply.Obj, supply.Brand, supply.Count.ToString(), addinfo);
        }
    }
}

```

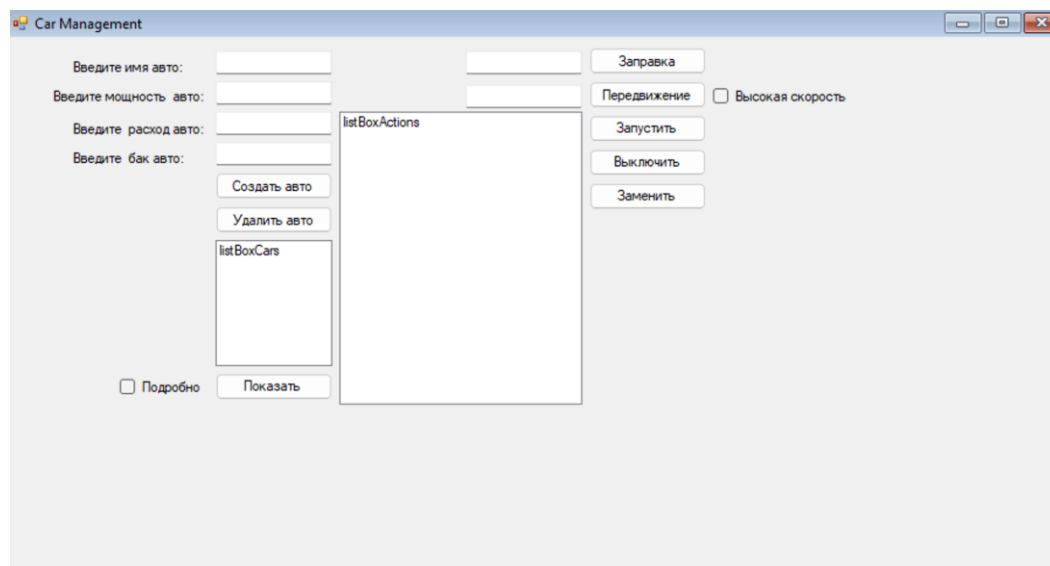


Рисунок 6 – Расположение графических элементов

5 Результат работы программы

На рисунке 7 представлены результаты работы программы.

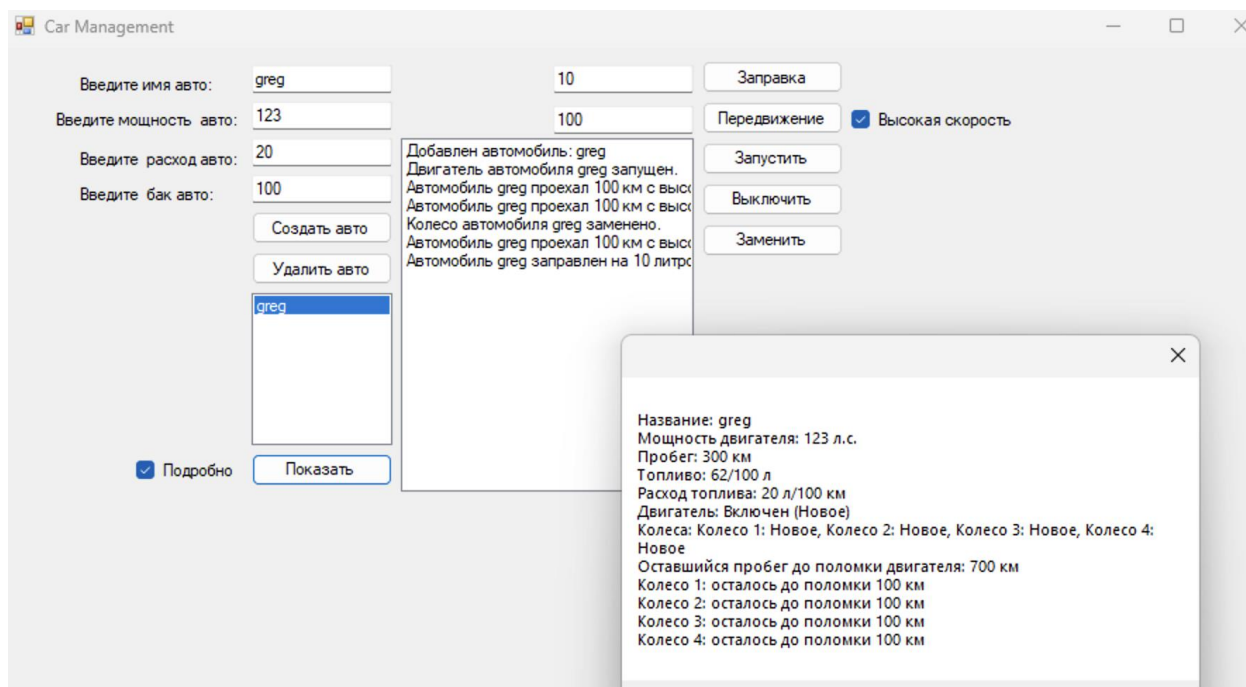


Рисунок 7 – Результат работы программы

6 Вывод

В этой лабораторной работе мной были изучены свойства классов `get` и `set` и реализованы в приложении Windows Forms.